Project 3 – DEX-NFT Pawning Dapp

Decentralized Computing and Blockchains

2025/2026

Lenny Briclet (66926)
Valentin Barner (66958)
May 26, 2026

1 Introduction

This report describes the implementation of a decentralized application (Dapp) that combines a DEX (Decentralized EXchange) marketplace, an NFT marketplace with fixed-price sales and auctions, and two lending mechanisms: one backed by DEX collateral and another backed by NFT collateral with a third-party DEX backer. The project follows the Solidity, Hardhat and ethers.js practices presented during the course. It satisfies the six functionalities listed in the project statement and respects the rule that all financial logic must reside on-chain, while off-chain state is restricted to NFT metadata such as names, descriptions and image URLs.

2 Architecture

The system is split between three on-chain smart contracts (Solidity 0.8.28) and two off-chain components, as shown in Figure 1.

- **DexToken** (ERC20) implements the DEX fungible token and the buy/sell operations against ETH at a configurable rate.
- **NftCollection** (ERC721) lets users mint, burn and look up NFTs. It also stores an on-chain value used by the lending logic.
- **PawningHub** is the orchestrator. It coordinates loans, marketplace listings, auctions and the administrator console. After deployment its ownership of DexToken allows the admin to change parameters from a single entry point.
- A small **Express server** on port 3001 stores NFT metadata (name, description, image URL) only. It holds no funds and makes no financial decision.
- A static **web frontend** (HTML + ethers.js v5) connects through MetaMask, calls the three contracts and queries the metadata server.

The Hardhat network (chain ID 31337) is used for local testing and demonstration.

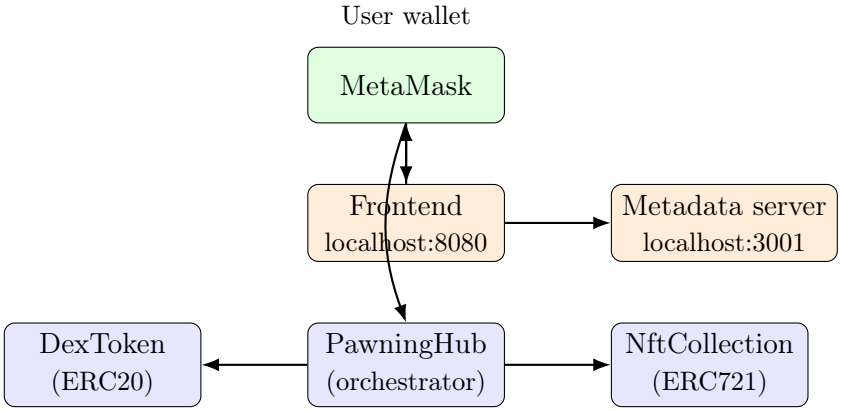

Figure 1: System architecture: three on-chain contracts (blue) plus two off-chain components (orange).

3 Functionalities

3.1 DEX Marketplace

DexToken initialises a fixed pool of DEX held by the contract itself and exposes a swap rate measured in wei per DEX. The `buyDex()` function sends `msg.value / dexSwapRate` DEX to the caller and `sellDex(amount)` performs the reverse. The contract is funded with ETH during deployment so users can withdraw value when selling.

3.2 Ether Loans with DEX Collateral

A borrower opens a loan with `loanDex(dexAmount, duration)`. The hub escrows the collateral, applies a 50% loan-to-value ratio and transfers ETH to the borrower. The duration is split into payment cycles; each cycle the borrower pays a fixed interest slice through `makeDexPayment`. A missed cycle automatically triggers `_liquidateDexLoan` and the collateral is transferred to the protocol. The borrower can close the loan early using `terminateDexLoan`, which charges a fixed termination fee in addition to the remaining interest.

3.3 NFT Marketplace

NftCollection lets any user mint NFTs through `mint(uri, valueInWei)` and destroy them with `burn(tokenId)`. The URI points to the metadata server. Sellers list NFTs for a fixed price using `listFixed`; buyers pay with either ETH or DEX. The hub performs the conversion using the current swap rate, so a listing priced in one currency can be paid in the other.

3.4 NFT Auctions

`listAuction` creates a time-bound listing with a minimum price and a maximum wait. Each call to `bid` must exceed the previous bid; the previous bidder is automatically refunded. After the deadline anyone calls `finalizeAuction`: the NFT moves to the highest bidder and the funds move to the seller. If no bid was placed, the NFT is returned to the seller.

3.5 NFT-Backed Loans with DEX Backer

The platform cannot own arbitrary NFTs, so NFT-backed loans rely on a third actor, the *backer*. The borrower calls `requestNftLoan`; the NFT is escrowed. A backer then calls `fundNftLoan`, supplying the required DEX, and the hub releases ETH to the borrower (50% of the NFT's on-chain value). Half of every interest payment is forwarded to the backer through `makeNftPayment`. On normal closure, the backer recovers the DEX and half of the remaining interest plus half of the termination fee. On default, both the NFT and the DEX backing are transferred to the backer; this is the incentive that makes such loans possible.

3.6 Administrator Console

PawningHub exposes `onlyOwner` setters for the payment cycle, interest percentage, termination fee, maximum loan duration and DEX swap rate, plus withdraw functions for ETH and DEX held by the contract. The frontend shows an Administrator tab only when the connected account equals `hub.owner()`.

4 Security and Design Choices

All money handling lives in the smart contracts. Every external function that moves value uses OpenZeppelin's `ReentrancyGuard`, `SafeERC20` wrappers and explicit access control through `Ownable`. State updates always precede external calls (Checks-Effects-Interactions). Liquidations are deterministic so no off-chain oracle is required. The metadata server only serves and persists JSON; an outage or corruption there never affects on-chain balances. The frontend uses MetaMask for signing and verifies the network (chain ID 31337) before any transaction, recreating the provider when the chain changes to avoid network-mismatch errors.

5 Running and Testing

A `README.md` and a deploy script automate the setup. Four terminals are required: the Hardhat node, the deployment script (which seeds liquidity and copies the ABIs to the frontend), the metadata server and the static frontend served on port 8080. The deployment also transfers ownership of `DexToken` to `PawningHub` so requirement 6 can be enforced from a single contract. The contract suite is covered by 27 Hardhat tests organised by feature (`dex`, `nft`, `hub-dex-loan`, `hub-market`, `hub-nft-loan`, `hub-admin`); all of them pass. The full system runs locally on Hardhat; deploying to a public testnet only requires changing the network configuration.